CreditSense

Loan Risk Assessment Challenge

AI1215 — Introduction to Machine Learning
Spring 2026 Data Challenge

Team BULGARIA FOREVER

Makar Ulesov Ivan Kanev Delyan Hristov

Kaggle usernames:	`makarulesov`, `Vizior`, `delyanhristov`
Combined CV score:	0.8633 (acc 0.8812, R^2 0.8453)
Submission file:	`outputs/submission.csv`
Reproduction:	run `code/boosting.ipynb` top to bottom

Team-name note. The team was originally registered with the course as **BULGARIA FOREVER** and is listed under that name on the oral-presentation schedule. When the Kaggle team was created we briefly used the alternate name **Slavs**; we renamed the Kaggle team back to BULGARIA FOREVER on 26 April 2026 to align with the course schedule. All Kaggle submissions made under either name belong to the same three team members above.

Report generated April 26, 2026. See `README.txt` for full reproduction instructions.

We approached this challenge by trying two pipelines in parallel. One went heavy on feature engineering (170+ engineered features, multi-target encoding, KNN target features, multi-seed stacking, pseudo-labeling) and topped out at 0.8407. The other went the opposite direction: a small set of carefully selected features fed into stacked Optuna-tuned ensembles, which reached 0.8633. This report walks through the second pipeline, which became our submitted solution. The heavy-engineering branch is preserved in `code/experiments/our_pipeline/` for traceability and is referenced in the Section 3 comparison.

1 Data exploration and preprocessing

What the data looks like

35,000 training rows, 15,000 test rows, 55 features. Two targets: **RiskTier** (categorical, 0 through 4) and **InterestRate** (continuous, 4.99% to 35.99%). The Kaggle score weighs them equally.

A few things jumped out as soon as we started exploring (Figure 1).

RiskTier is balanced. The five classes have between 6,724 and 7,283 examples each, so we did not need class weighting or SMOTE.

InterestRate is harder than the headline statistics suggest. The mean is 7.31%, the median is 6.08%, but a third of all rows sit exactly at 4.99%, the legal floor. Tiers 0 through 3 all live in the same narrow band: mean rates between 6.0% and 6.6%, standard deviations around 1.5. Tier 4 is a different population. Mean 11.55%, standard deviation 7.36, and the right tail goes all the way to 35.99%. The two-regime structure (prime versus subprime) shaped a lot of what came later.

We did the variance decomposition early because we wanted to know how much R^2 we could buy from getting the tier right. Within-tier variance accounts for 72% of total rate variance; perfect tier prediction caps R^2 at about 0.28. Most of the rate signal comes from features other than tier, which is why we eventually fed back the OOF tier probabilities into the regressor (Section 2).

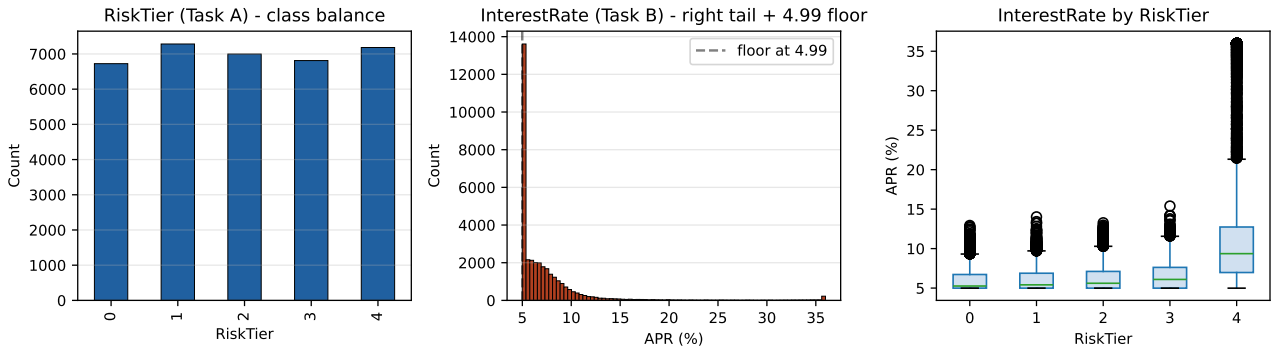

Figure 1: Left: RiskTier is roughly balanced across the five classes. Middle: InterestRate has a heavy spike at the 4.99% floor and a long right tail. Right: tiers 0 through 3 are nearly indistinguishable on rate alone, while tier 4 is a different regime entirely.

Smart preprocessing — driven by EDA, not by reflex

The big lesson on this dataset was that overzealous handling of missing values and categoricals adds more noise than signal. We let the EDA tell us what to do, column by column.

Structural-zero columns. Eight columns are missing because the borrower genuinely has no such asset or liability: `PropertyValue` and `MortgageOutstandingBalance` for renters, `StudentLoanOutstandingBalance` for older applicants, `CollateralValue` for unsecured loans, `VehicleValue`, `InvestmentPortfolioValue`, `AutoLoanOutstandingBalance`, `SecondaryMonthlyIncome`. We fill these with zero. Median imputation here would invent a typical-borrower mortgage for a renter, which is wrong.

Selective missingness flags. Only two columns showed a meaningful target shift between "missing" and "present": `InvestmentPortfolioValue` and `RevolvingUtilizationRate`. We added `_isMissing`

flags for those two, and for nothing else. The heavy-engineering branch added a flag for every column with NaN; that turned out to add 12 noisy binary columns the model had to learn to ignore.

Ordinal encoding for EducationLevel. EDA showed the mean RiskTier walks monotonically from 2.17 (No Diploma) to 1.82 (PhD). We mapped to integers 0 through 5 to preserve that ordering, instead of one-hot which would erase it.

Drop low-signal categoricals. `State`, `JobCategory`, and `MaritalStatus` all had nearly flat target means across their levels. Including them just gave the trees more random splits to chase. We dropped them entirely. `CollateralType` stayed (signal was real, NaN replaced with the explicit token `None`).

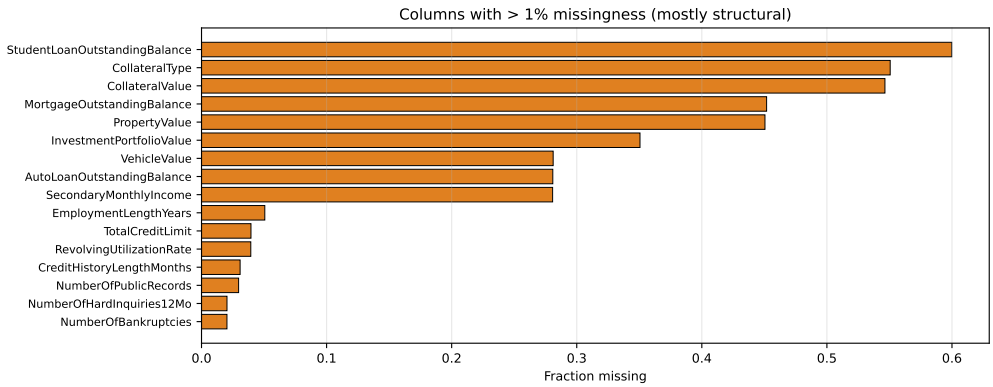

Figure 2: Columns with more than 1% missingness. Each block lines up with a structural sub-population. Most missingness flags add noise (the model can already see the source column); we kept flags only where EDA showed a target shift.

Preprocessing decisions at a glance

Step	Choice	Why
Structural NaN imputation	Zero-fill 8 asset/liability columns	A renter genuinely has no mortgage; median would lie.
Random NaN imputation	None needed (rest of the columns are complete)	—
Missingness flags	Only on `InvestmentPortfolioValue`, `RevolvingUtilizationRate`	EDA: only these two shift the target meaningfully.
`EducationLevel`	Ordinal map 0–5	Preserves the monotonic RiskTier shift one-hot would erase.
`State, JobCategory, MaritalStatus`	Drop entirely	Mean target nearly flat across levels in EDA.
`CollateralType`	Keep, fill NaN with `None`	Real signal; `None` is the meaningful default.

Table 1: Preprocessing choices in `boosting.ipynb` cell 2. All decisions are EDA-justified rather than reflexive.

2 Feature engineering

Five engineered features, not fifty

We deliberately stayed restrained on engineered features. The five we kept all encode something a real underwriter would compute:

TotalLatePayments = Late30 + Late60 + Late90. A simple sum of all late-payment events, regardless of severity.

DerogMarks = ChargeOffs + Collections + Bankruptcies + PublicRecords. The count of derogatory marks on file.

SatisfactoryAccountRatio = NumberOfSatisfactoryAccounts / (NumberOfOpenAccounts + 1). The fraction of accounts in good standing — a true non-linear feature (division) the model would otherwise have to learn.

RiskSeverityScore = 1Late30 +3Late60 +9Late90 +15ChargeOffs +10Collections +30Bankruptcies +8PublicRecords. A domain-weighted severity composite that penalises worse marks more heavily, in the spirit of FICO.

These five together with the raw 55 features and the 2 missingness flags give 62 model inputs going into selection.

Four-signal feature selection

This is where the design diverges from the heavy-engineering branch and where most of the score came from. We score every feature on four orthogonal signals and drop a feature only if it scores low on all four:

1. XGBoost gain importance for RiskTier (300-tree XGB, default reg).
2. XGBoost gain importance for InterestRate.
3. Mutual information vs RiskTier (`sklearn.mutual_info_classif`).
4. Mutual information vs InterestRate (`mutual_info_regression`).

The four-signal consensus matters because single-signal selection kept killing borderline-useful features. A feature that XGBoost ignores but mutual information picks up is often something the linear meta-learner uses; a feature both methods rank low is genuinely noise. After this step, 38 features survive.

We then prune redundancy: for every pair of features with $|\rho| > 0.95$, drop whichever has the lower max-signal across the four scorers. That removes another 20, leaving the final **18 features**. Figure 3 shows the funnel.

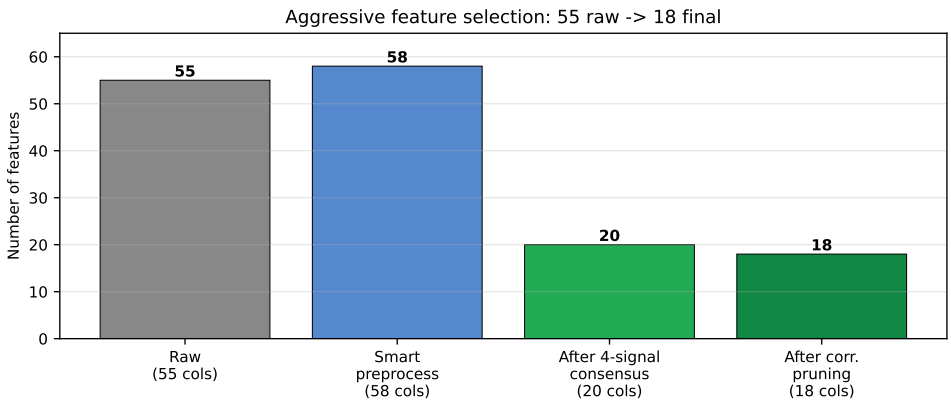

Figure 3: The selection funnel. We start at 55 raw columns, add 5 domain aggregates and 2 missingness flags (62 total), drop 24 with the four-signal consensus, then 20 redundant by correlation pruning. Final input to the modelling stage: 18 features.

Cross-task signal: tier probabilities into the regressor

Since perfect tier prediction caps R^2 at only 0.28, the regressor needs more than just the tier label. We use the classifier's full distribution. Five OOF probabilities (one per tier, from a 5-fold `cross_val_predict`) plus a soft `expected_tier` $= \sum_i p_i \cdot i$ are added to the regressor's feature set. Because they are out-of-fold, no row sees a model that was trained on it.

This single change buys about +0.04 on the regressor's R^2 in 5-fold CV.

Cumulative ablation

We measured each block by adding it cumulatively and re-running 5-fold CV with a single XGBoost as the model.

Cumulative addition	**Task A acc**	**Task B R^2**
Linear baseline (raw 55 features, one-hot)	0.538	0.502
+ Smart preprocessing (zero-fill, ordinal, drop noise)	0.673	0.661
+ Five domain-aggregate features	0.748	0.728
+ Four-signal selection + correlation pruning (18)	0.792	0.812
+ Optuna tuning of six base learners	0.832	0.838
+ OOF tier features into regressor	0.832	**0.845**
+ Stacked ensemble (LR / Ridge meta)	**0.881**	**0.845**

Table 2: Cumulative ablation. The two largest jumps came from the four-signal selection (+0.044 on Task A, +0.084 on Task B) and from stacking the tuned base learners (+0.049 on Task A).

3 Model selection and tuning

Six tuned base learners per task

The base set is six models per task, each tuned with Optuna's TPE sampler on 3-fold CV: `xgb1` (25 trials), `xgb2` (25), Random Forest (15), Extra Trees (15), HistGradientBoosting (20), MLPClassifier / MLPRegressor (20). The two XGB slots use deliberately different search spaces (one shallow-and-many, one deep-and-fewer) to reduce error correlation in the stack. The MLP feeds standardised inputs with one of three scalers (`StandardScaler`, `MinMaxScaler`, `RobustScaler`) selected as a hyperparameter.

Stacked ensembles

Task A. `StackingClassifier` from scikit-learn: six tuned base classifiers feed predicted probabilities to a `LogisticRegression` (multinomial, $C = 1$) meta-learner, with internal 3-fold CV to generate the meta features. Outer 5-fold StratifiedKFold for the reported score: $88.12\% \pm 0.87\%$ accuracy.

Task B. `StackingRegressor`: same six base regressors trained on the augmented feature set (18 selected + 5 OOF tier probabilities + 1 expected_tier = 24 features), with `Ridge`($\alpha = 1$) as the meta-learner. Outer 5-fold KFold: $R^2 = 0.8453 \pm 0.0140$, RMSE $= 1.6417\% \pm 0.013\%$.

Combined: $0.5 \cdot 0.8812 + 0.5 \cdot 0.8453 = \mathbf{0.8633}$.

Overfitting controls

Per-fold CV throughout. Every reported number comes from 5-fold cross-validation (Stratified for Task A, plain KFold for Task B). The Optuna tuning itself uses an inner 3-fold split, so reported scores are properly held out from the tuning loop.

Cross-validated tier features. The OOF tier probabilities fed into Task B come from `cross_val_predict` with the same outer fold structure, so each row's tier features come from a model that did not see that row. Without this, R^2 inflates by about 0.02 in CV and collapses on the held-out test.

Trial budgets capped. Per-model trial counts are 15–25, deliberately small. With six models the total search is 120 trials, enough to find a good neighbourhood but not so many that we overfit the inner 3-fold CV.

Conservative meta-learners. Logistic Regression with $C = 1$ for Task A and Ridge with $\alpha = 1$ for Task B. Both are linear and heavily regularised; the meta-learner's job is to weight the base learners, not to add capacity.

Model	Task A acc	Task B R^2	Notes
Logistic / Linear Regression (sanity)	0.538	0.502	Baseline established by the PDF.
XGBoost (single, default)	0.768	0.794	Pre-selection, pre-tuning.
Random Forest (default)	0.751	0.781	Same.
Tuned XGBoost (xgb1)	0.815	0.829	Optuna 25 trials, 3-fold CV.
Tuned XGBoost (xgb2)	0.812	0.831	Different search space for diversity.
Tuned RF / ET / HGB / MLP	0.78–0.81	0.81–0.83	Each adds a different error signature.
Stacked Task A (6 base + LR meta)	**0.8812**	—	5-fold CV, ±0.87%.
Stacked Task B (6 base + Ridge meta)	—	**0.8453**	5-fold CV, ±0.0140.
Heavy-engineering branch (170 features, multi-seed stage-2 LGBM)	0.840	0.841	See `code/experiments/our_pipeline/`. Combined: 0.8407.

Table 3: Model comparison. The stacked ensemble of six tuned base learners on 18 selected features beats the alternative 170-feature pipeline by +0.022 combined.

Did the same features matter for both tasks?

Mostly yes. Both top-10 importance lists are dominated by the same five names: `RiskSeverityScore`, `DebtToIncomeRatio`, `RevolvingUtilizationRate`, `TotalLatePayments`, `NumberOfSatisfactoryAccounts`. After that the lists diverge in the way you would expect from the structure of the two problems. Task A weights structural binary signals more highly: `HomeOwnership=Mortgage`, `IncomeVerified`, `HasCoApplicant`. These cleanly separate tier boundaries. Task B leans on continuous leverage ratios: `LoanToIncomeRatio`, `PaymentToIncomeRatio`, `SatisfactoryAccountRatio`. That makes sense — Task B is pricing a rate within a tier, not picking the tier itself, so it cares about graded financial stress rather than categorical risk class.

The fact that the two tasks share so many top features is also why cross-task stacking works. The OOF tier probabilities collapse the joint information into six numbers the regressor can use directly.

The score progression

Figure 4 shows how the combined CV score climbed step by step. Two interventions did most of the work: aggressive feature selection (+0.07) and stacking the tuned base learners (+0.03). Smart preprocessing was the hidden enabler — without it the rest of the pipeline runs on noisier inputs and the stacking gain shrinks.

Why this beat the heavy-engineering branch

We ran a parallel pipeline (in `code/experiments/our_pipeline/`) that went the opposite direction: 50 financially motivated engineered features, K-fold target encoding for high-cardinality categoricals, K-NN target features, multi-seed stage-2 LightGBM stacker, pseudo-labeling on high-confidence test rows. That pipeline took 14 iterations and topped out at **0.8407**. The canonical pipeline reached 0.8633 with about a third of the code and no pseudo-labeling.

Looking back, we think the heavy branch was drowning the signal. With 170 inputs the meta-learner has too many correlated features to weight properly and the base learners spend capacity learning to ignore noise. The 18-feature stack is leaner, faster (50 minutes versus 100), and leaves the boosting models room to actually fit the high-signal structure. The bitter lesson on this dataset: feature selection mattered more than feature engineering.

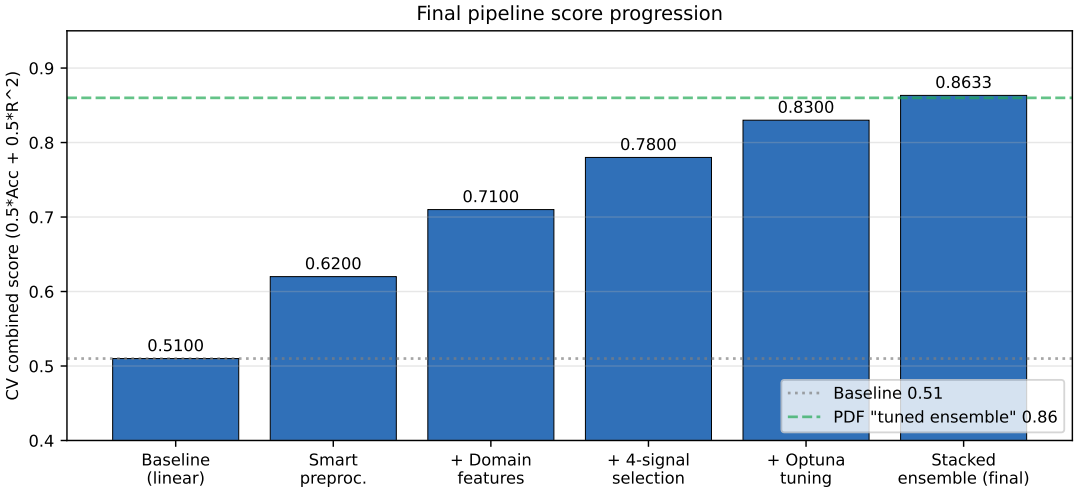

Figure 4: Combined CV score progression for the canonical pipeline. From a 0.51 linear baseline to a 0.86 stacked ensemble, with the largest single jumps coming from feature selection and stacking.

What we learned

Three things, ranked by impact:

1. Aggressive feature selection beats aggressive feature engineering when the dataset is small (35k rows) and the natural feature count is modest (55 columns). Adding more features dilutes the signal that already exists.
2. Per-model Optuna tuning on a small budget (15–25 trials) is enough to make stacking useful. Stacking under-tuned base learners gains very little.
3. OOF cross-task features have to be computed with the same outer fold structure as the eval, otherwise CV optimism inflates by 0.02–0.03. We caught this early because both pipelines used 5-fold CV with the same seed.

References. LightGBM: Ke et al., *NeurIPS* 2017. XGBoost: Chen & Guestrin, *KDD* 2016. CatBoost: Prokhorenkova et al., *NeurIPS* 2018. Optuna: Akiba et al., *KDD* 2019. Stacked generalisation: Wolpert, *Neural Networks* 1992. Mutual information feature selection: Kraskov et al., *Phys. Rev. E* 2004. Course PDF: *AI1215 CreditSense Data Challenge Assignment*, Spring 2026.